Docker container – tutorial – p1

Nice tutorial for Mac and Linux
https://www.youtube.com/watch?v=JprTjTViaEA

Download docker for Mac – file Docker.dmg :
https://hub.docker.com/editions/community/docker-ce-desktop-mac

Install it – now you have Docker.app in Applications
Run it, a docker icon will appear on the top bar of Mac desktop.
It will ask to login using your login/password at docker.com
Now you can open terminal and run docker commands:

Official docker tutorial:
- https://docs.docker.com/get-started/

One more tutorial:
- https://medium.com/@patrickmichelberger/getting-started-with-anaconda-docker-b50a2c482139

This tutorial:
- https://jonnylangefeld.github.io/learning/Docker/How%2Bto%2BDocker.html

Reference for "dicker run":
- https://docs.docker.com/engine/reference/run/

Reference for Dockerfile:
- https://docs.docker.com/engine/reference/builder/

```
docker --version
docker run hello-world
docker ps –a   # shows all running and exited containers
docker run --name my-hello hello-world
docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS                      PORTS               NAMES
7f02d0054824        hello-world         "/hello"            6 seconds ago       Exited (0) 5 seconds ago                        my-hello
877d44086bed        hello-world         "/hello"            4 minutes ago       Exited (0) 4 minutes ago                        gifted_Sutherland

docker run -it --name my-linuc-container ubuntu bash
…
root@05c21049cf30:/#

Try running commands on prompt:  top , pwd , ls –alF , etc., then exit

docker ps –a   # now shows 3 exited containers

docker run -it --name my-linuc-container-1 ubuntu bash
docker run -it --name my-linuc-container-2 ubuntu bash

# Make a list of IDs of all exited containers:
docker ps -a -f status=exited -q

# Remove them from list:
docker rm $(docker ps -a -f status=exited -q)
```

Docker container – tutorial – p2

Continue tutorial - https://www.youtube.com/watch?v=JprTjTViaEA
==========
Add local directory as volume into the container:

```
docker run -it --name my-linux-container --rm -v $HOME/test:/my-data  ubuntu  bash
```

==========
Creating your own container

```
vi Dockerfile

FROM ubuntu
CMD echo "Hello Jonny" # some comment

docker build -t my-ubuntu-image .
docker images
docker run my-ubuntu-image
```

==========
Another example:

```
FROM ubuntu
RUN apt-get upgrade && apt-get update && apt-get install -y python3
```

==========
Another example – use anaconda python from https://hub.docker.com/u/continuumio/

```
docker pull continuumio/anaconda3
docker run -i -t continuumio/anaconda3 /bin/bash

docker pull continuumio/miniconda3
docker run -i -t continuumio/miniconda3 /bin/bash
```

You can start a Jupyter Notebook server and interact with anaconda3 or Miniconda via your browser:

```
docker run -i -t -p 8888:8888 continuumio/miniconda3 /bin/bash -c "/opt/conda/bin/conda install jupyter -y --quiet && mkdir /opt/notebooks &&
/opt/conda/bin/jupyter notebook --notebook-dir=/opt/notebooks --ip='*' --port=8888 --no-browser"
```

You can then view the Jupyter Notebook by opening http://localhost:8888 in your browser,
or http://<docker-machine-ip>:8888 if you are using a Docker Machine VM.
==========
Here is how to save a docker image to a file:

```
docker save –o myimage.tar myimage1 myimage2
docker save myimage > myimage.tar
docker save myimage | gzip > myimage.tar.gz
```

Docker container – tutorial – p3

Recap: Docker commands

```
docker build -t friendlyname .  # Create image using this directory's Dockerfile
docker run -p 4000:80 friendlyname  # Run "friendlyname" mapping port 4000 to 80
docker run -d -p 4000:80 friendlyname         # Same thing, but in detached mode
docker ps                                  # See a list of all running containers
docker stop <hash>                      # Gracefully stop the specified container
docker ps -a           # See a list of all containers, even the ones not running
docker kill <hash>                    # Force shutdown of the specified container
docker rm <hash>              # Remove the specified container from this machine
docker rm $(docker ps -a -q)           # Remove all containers from this machine
docker images -a                               # Show all images on this machine
docker rmi <imagename>            # Remove the specified image from this machine
docker rmi $(docker images -q)             # Remove all images from this machine
docker login             # Log in this CLI session using your Docker credentials
docker tag <image> username/repository:tag  # Tag <image> for upload to registry
docker push username/repository:tag            # Upload tagged image to registry
docker run username/repository:tag                   # Run image from a registry
```

Docker container – tutorial – p4

Docker runs in its own VM. And it stores images inside it.
For example, this file is approx. 64 GB on my Mac:
$HOME/Library/Containers/com.docker.docker/Data/vms/0/data/Docker.raw

```
# Clean commands (for Linux, docker v.19.x):
# remove exited containers:
docker ps --filter status=dead --filter status=exited -aq | xargs -r docker rm -v
# remove unused images:
docker images --no-trunc | grep '<none>' | awk '{ print $3 }' | xargs -r docker rmi
# remove unused volumes:
docker volume ls -qf dangling=true | xargs -r docker volume rm
```

```
# Clean commands (for Mac, docker v.19.x – no “-r” option for xargs):
# remove exited containers:
docker ps --filter status=dead --filter status=exited -aq | xargs    docker rm -v
# remove unused images:
docker images --no-trunc | grep '<none>' | awk '{ print $3 }' | xargs    docker rmi
# remove unused volumes:
docker volume ls -qf dangling=true | xargs    docker volume rm
```

Docker container – tutorial – p5

```
# stop running containers:
# Use command “docker ps” to show only running containers
# Add option “-q” (quiet) to only display numeric IDs
docker stop `docker ps -q`

# when mapping volume or port, we use syntax:    external:internal
docker run –p 5000:80 –v /my_real_path/:/path_inside_docker/  myapp


# example building current directory – and running it as daemon ( -d option):
docker stop `docker ps -q`
docker build . –t myapp && docker run –v /out_path/:/in_path/ –p 5000:80 –d myapp


# login inside a running container:
docker ps                                  # get ID of your container
docker exec –it 37701c34dfa9 /bin/bash     # login into your container
```

Docker container – tutorial – p6

```
# some common Dockerfile commands – see more here https://docs.docker.com/engine/reference/builder/

# Comment
RUN echo 'something'
RUN mv file1 file2
COPY file1 file2
RUN mkdir -p /var/log/mydir
RUN apt-get update && apt-get install –y ssmtp && apt-get clean

CMD ...   # command to execute when container starts

EXPOSE 80
EXPOSE 443
```

Docker container – tutorial – 7

Running multiple services inside one container https://docs.docker.com/config/containers/multi-service_container/

The simplest way to start several services inside a container is to copy them from outside – and then start with a wrapper script. The drawback of this approach is that we will need to install these services manually on the box and to know all directories to copy.

```
FROM ubuntu:latest
COPY /out_path/my_services          /in_path/my_services
...
COPY /out_path/my_wrapper_script.sh ./my_wrapper_script.sh
CMD ./my_wrapper_script.sh
```

Another approach would be to install everything inside the container:

```
FROM continuumio/anaconda3
RUN apt-get update && apt-get install –y nginx
RUN apt-get update && apt-get install –y gunicorn
COPY ...
...
CMD  ...
```

Yet another approach is to use ”Docker Compose” to use multiple containers together:
https://docs.docker.com/compose/

Docker container – tutorial – 8

Using ssh inside container: https://stackoverflow.com/questions/18136389/using-ssh-keys-inside-docker-container

```
==========================================================
Build Context – files and directories which docker can copy into image during the build process.
Build context is not required for for running the image.
==========================================================
A Docker image consists of read-only layers each of which represents a Dockerfile instruction.
The layers are stacked, and each one is a delta of the changes from the previous layer.
    FROM ubuntu:18.04        # creates a layer from the ubuntu:18.04 Docker image.
    COPY . /app              # adds files from your Docker client’s current directory.
    RUN make /app            # builds your application with make.
    CMD python /app/app.py   # specifies what command to run within the container.
==========================================================
# setting environment usint “ENV” command
    ENV PATH /usr/local/nginx/bin:$PATH

Each next layer inherits from previous layers – unless you unset (in the same layer):

    FROM alpine
    RUN export ADMIN_USER="mark" && echo $ADMIN_USER > ./mark && unset ADMIN_USER
    CMD sh

docker run --rm test sh -c 'echo $ADMIN_USER'
```

Docker container – tutorial – 9

How can Docker supply each container with a hostname and DNS configuration, without having to build a custom image with the hostname written inside?
- https://docs.docker.com/v17.09/engine/userguide/networking/default_network/configure-dns/

How to tell docker which DNS servers to use:
- https://l-lin.github.io/post/2018/2018-09-03-docker_ubuntu_18_dns/

How to tell docker about external host (to make it visible inside docker):
- https://blog.codeship.com/using-the-add-host-flag-for-dns-mapping-within-docker-containers/

Creating/using nginx docker (also how to configure)
- https://docs.nginx.com/nginx/admin-guide/installing-nginx/installing-nginx-docker/

How do I combine several images into one via Dockerfile
- https://github.com/moby/moby/issues/3378

Multiple FROMs - what it means
- https://stackoverflow.com/questions/33322103/multiple-froms-what-it-means
- https://dev.to/brpaz/using-docker-multi-stage-builds-during-development-35bc
- https://medium.com/@tonistiigi/advanced-multi-stage-build-patterns-6f741b852fae

Brief history of containers:
- https://blog.aquasec.com/a-brief-history-of-containers-from-1970s-chroot-to-docker-2016

Docker container – tutorial – 10

```
How to schedule jobs defined in containers
Choice of schedulers:
   cron, systemd, Ofelia, etc.
```

- https://devops.stackexchange.com/questions/4904/scheduling-a-docker-container-or-installing-scheduler-in-container
- https://nickjanetakis.com/blog/docker-tip-40-running-cron-jobs-on-the-host-vs-in-a-container
- https://www.tddapps.com/2017/02/18/how-to-run-any-container-on-a-schedule/
- https://blog.trifork.com/2014/03/11/using-supervisor-with-docker-to-manage-processes-supporting-image-inheritance/
- https://coreos.com/os/docs/latest/scheduling-tasks-with-systemd-timers.html
- https://github.com/mcuadros/ofelia